

Você Estava *Online*? Reconstruindo Sessões de Usuários em Sistemas de Larga Escala na Internet

Weverton Luis da Costa Cordeiro^{1,2}, Rodrigo Brandão Mansilha¹, Flávio Roberto Santos¹, Luciano Paschoal Gaspar¹, Marinho Pilla Barcellos¹

¹Instituto de Informática – Universidade Federal do Rio Grande do Sul (UFRGS)
Porto Alegre – RS – Brasil
{wlccordeiro, rbmansilha, frsantos, paschoal, marinho}@inf.ufrgs.br

²Instituto Federal do Pará em Itaituba (IFPA)
Itaituba – PA – Brasil
weverton.cordeiro@ifpa.edu.br

Abstract. *The comprehension of users' behavior is paramount for evaluating improvements to networked, distributed systems. To this end, several strategies have been proposed to obtain traces, based on the capture of usage information, which can then serve for evaluation purposes. One main strategy consists of taking snapshots of online users, using instrumented clients. In spite of its popularity, related proposals have fallen short in ensuring accuracy of obtained data. For a variety of reasons, users may fail to appear in some snapshots, although online. In this paper, we propose a methodology to correct ill-collected snapshots and build more accurate traces from them. In summary, we estimate the probability that a given snapshot is missing some users. The snapshot is corrected if the probability exceeds a given threshold. We use ground-truth data to assess the effectiveness of our methodology. The impact of our proposal is evidenced by means of an analysis of traces from a BitTorrent swarm.*

Resumo. *A compreensão do comportamento de usuários é fundamental para avaliar melhorias a sistemas em rede e distribuídos. Para este fim, várias estratégias tem sido propostas para obter traços, baseadas na captura de informações de uso, os quais podem então servir para fins de avaliação. Uma estratégia básica consiste em obter fotos (snapshots) dos usuários online, usando agentes instrumentados. Apesar de popular, as propostas relacionadas tem sido incapazes de garantir a acurácia dos dados obtidos. Por diversos motivos, os usuários podem não aparecer em alguns snapshots, embora online. Neste artigo é proposta uma metodologia para corrigir imprecisões em snapshots e construir traços precisos a partir deles. Em resumo, é estimada a probabilidade de que um snapshot falhou em “capturar” um usuário online. O snapshot é corrigido caso a probabilidade exceda um determinado limiar. A eficácia da metodologia é avaliada com base em um conjunto de dados de referência. O impacto da proposta é demonstrado em uma análise de traços de enxames BitTorrent.*

1. Introdução

Há um entendimento (explícito ou implícito) na comunidade de que é fundamental compreender o comportamento dos usuários em um determinado sistema antes de se propor melhorias ao mesmo. No âmbito de redes e sistemas distribuídos, esse entendimento tem motivado a investigação de modelos analíticos que permitam analisar o comportamento dos usuários em diversas subáreas; exemplos incluem recuperação de conteúdo em redes orientadas a conteúdos [Rosensweig et al. 2013] e mobilidade em redes celulares [Sridharan and Bolot 2013]. Nesse contexto, um dos maiores desafios científicos é capturar informações precisas sobre

sessões *online** de usuários em sistemas de larga escala na Internet. Esse desafio torna-se mais complexo devido à variedade de paradigmas de sistemas existentes, por exemplo redes par-a-par [Barcellos and Gaspary 2006], redes sociais [Teles et al. 2013], redes de sensores [Ruiz et al. 2006] e Internet das Coisas [França et al. 2011].

Há diversos esforços de pesquisa na direção de novas abordagens para obter traços com base na captura de informações sobre sessões *online* de usuários [Yoshida and Nakao 2011a, Hoäyfeld et al. 2011]. Além de ajudar na formulação de modelos analíticos, tais traços podem ser úteis como entrada para simulações. Em resumo, essas abordagens podem ser mapeadas em três estratégias básicas: 1) coleta de informações diretamente do banco de dados do sistema alvo [Guo et al. 2007]; 2) instalação de *plug-ins* nos dispositivos dos usuários [Alexa Internet, Inc. 2012]; e 3) usar monitores para capturar periodicamente fotos (*snapshots*) dos usuários, e usá-las para sintetizar traços [Hoäyfeld et al. 2011].

A terceira estratégia tem recebido bastante atenção [Yoshida and Nakao 2011b, Cuevas et al. 2010, Blond et al. 2010], o que pode ser explicado pela maior autonomia que ela oferece; ao contrário das demais, ela não requer acesso privilegiado a nenhuma entidade do sistema. Por esse motivo, neste artigo nos concentramos na terceira estratégia.

Definição do problema. Vários estudos tem mostrado as potencialidades da utilização da estratégia baseada em *snapshots* em uma série de sistemas de larga escala na Internet [Hoäyfeld et al. 2011, Mansilha et al. 2010]. Apesar de promissora, essa estratégia torna-se suscetível a imprecisões quando *snapshots* falham em capturar parte dos usuários *online* no sistema. Usuários *online* podem não aparecer em snapshots por limitações técnicas do sistema alvo. Em BitTorrent, um exemplo é o tamanho máximo das listas de pares, L_{max} . Caso o número de usuários for maior que L_{max} , alguns deixarão de ser reportados. Mesmo obtendo várias listas, ainda há a chance de que algum usuário não apareça em alguma. Isso representa um importante problema, posto que traços gerados com tais imprecisões podem levar a conclusões erradas sobre as características desejáveis para um sistema.

Contribuição. Neste artigo, nós abordamos o problema da construção de traços mais acurados a partir de um conjunto de *snapshots* imprecisos. Para esse fim, nós propomos uma metodologia que mensura a probabilidade de que um determinado *snapshot* é impreciso (ou seja, informa incorretamente que um usuário está *offline*) em um dado instante; caso a probabilidade supere um limiar estabelecido, o *snapshot* é corrigido. Nossa metodologia é agnóstica às propriedades específicas de sistemas e ao comportamento sazonal de usuários, podendo ser aplicada em diversos contextos. São apresentadas avaliações a respeito de como essa metodologia permite melhorar a precisão de traços de sessões *online* de usuários. Os traços resultantes (corrigidos) são analisados considerando métricas como recorrência e duração.

O restante deste artigo está organizado como segue. Na Seção 2 é apresentada uma discussão sobre o desafio enfrentado neste artigo e a ideia investigada para superá-lo. Na Seção 3 é descrito o modelo de sistema, premissas e limitações relacionadas à metodologia proposta. Na Seção 4 é introduzida a metodologia para identificação e correção de falhas em *snapshots* de sessões *online* de usuários. Na Seção 5 é avaliada a eficácia da solução enquanto que na Seção 6 é dada ênfase ao seu impacto na análise de traços reais de enxames BitTorrent. Por fim, na Seção 7 é discutida a literatura relacionada, ao passo que na Seção 8 são apresentadas as conclusões e trabalhos em andamento.

2. Contextualização e Motivação

A seguir apresentamos uma visão geral do *modus operandis* da estratégia abordada neste artigo, usando a Figura 1 como base. Nesta figura, os rótulos $v_{1..3}$ representam os usuários que foram

*No contexto deste artigo, entende-se por sessão *online* o período de tempo compreendido entre a chegada do usuário no sistema (i.e., *join* ou *login* do usuário no sistema) e a sua saída (i.e., *leave* ou *logout*). Outro aspecto que caracteriza uma sessão de usuário é o instante em que esses eventos de chegada e saída ocorreram.

vistos pelo menos uma vez no sistema; os rótulos $s_{1..14}$ representam os *snapshots* capturados, em intervalos regulares Δt , pelos monitores. Na figura, 1 indica que um determinado usuário v_i foi visto no sistema quando o *snapshot* s_t foi capturado (um *snapshot* “positivo”), enquanto que 0 (um *snapshot* “nulo”) representa o contrário. Construir o traço das sessões *online* (dos usuários) com base na informação representada na Figura 1 é uma tarefa trivial; para ilustrar, as sessões *online* identificadas nesse conjunto de *snapshots* é destacado em cinza na figura.

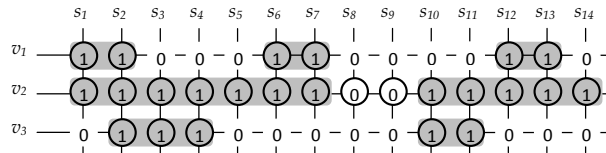


Figura 1. Usuários (v_i) vistos online em *snapshots* (s_t)

A Figura 1 ilustra também o problema abordado neste artigo. Considere que círculos na figura indicam que o usuário efetivamente estava *online* no sistema no instante da captura do *snapshot*. Note que os *snapshots* s_8 e s_9 falharam em capturar o usuário v_2 . Nesse caso, o traço resultante indicará duas sessões para v_2 , enquanto que na realidade o mesmo teve apenas uma.

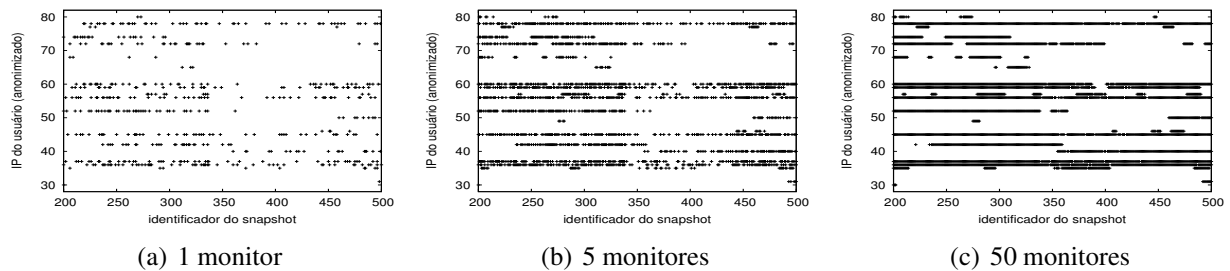


Figura 2. Conjunto de *snapshots* apresentado em três diferentes resoluções

Para mitigar esse problema, uma alternativa seria aumentar a quantidade de recursos (no caso, monitores) empregados na captura de *snapshots*. Como cada monitor é capaz de “fotografar” apenas uma fração dos usuários *online*, a precisão do traço resultante torna-se diretamente proporcional à quantidade de monitores utilizados; esse aspecto é ilustrado na Figura 2. Uma direção proposta por Zhang et al. [Zhang et al. 2011] para resolver esse problema é super-dimensionar a quantidade de monitores usados para “fotografar” o sistema. Portanto, um desafio adicional abordado neste artigo é melhorar a resolução de traços sem, no entanto, aumentar a quantidade de recursos necessária para se obtê-los.

3. Modelo e Notação do Sistema

Nesta seção são apresentados o modelo de sistema e a definição de *snapshot* adotados. Além disso, são discutidas premissas e limitações relacionadas à metodologia proposta.

3.1. Componentes e Operação do Sistema

Seja $G = \langle V_t, E_t, B \rangle$ um sistema formado por nós $V_t = \{v_1, \dots, v_m\}$, arestas $E_t \subseteq V_t \times V_t$ e um conjunto de *bootstraps* $B = \{b_1, \dots, b_n\}$. Cada nó corresponde a um usuário *online* no sistema no instante t . As arestas indicam ciência de presença *online*: uma aresta de v_i para v_j indica que v_i está ciente que v_j está *online* no instante t . Por “ciente” entende-se que v_i se comunicou com v_j recentemente, ou que v_i aprendeu essa informação de outra entidade (*bootstrap* ou usuário). O *bootstrap* é responsável pela admissão/registro de usuários no sistema. Assume-se que b_k é ciente sobre a presença de v_i no sistema no instante t se e somente se b_k foi responsável pela

admissão/registro de v_i ou v_i contactou b_k (por outro motivo). Seja $e(v_i) \subseteq V_t$ ($e(b_k) \subseteq V_t$) o conjunto de usuários *online* sobre os quais v_i (b_k) possui conhecimento.

Obter lista de pares. Essa requisição, lançada por v_i para b_k (ou v_j), consiste em obter uma (sub)lista L dos usuários *online* que b_k (ou v_j) está ciente; $L(v_i, x)$ denota a (sub)lista obtida por v_i de x (x pode ser b_k ou v_j). O tamanho máximo de tal lista é L_{max} . Note que v_i pode consultar qualquer usuário v_j desde que v_i esteja ciente de v_j . Por definição, $L(v_i, b_k) \subseteq e(b_k)$; similarmente, $L(v_i, v_j) \subseteq e(v_j)$.

3.2. Construção de Traço

Seja $\mathcal{S}$ um conjunto de *snapshots* dos usuários em G , coletados em um dado período. Um *snapshot* $s_j \in \mathcal{S}$ é uma lista dos usuários vistos *online* no instante t_j . Um *snapshot* é capturado a cada Δt unidades de tempo, nos instantes $t_0, t_1, \dots$, de modo que $t_j = t_0 + j \cdot \Delta t$. Seja V o conjunto dos nós vistos *online* pelo menos uma vez no sistema; por definição, $V_t \subseteq V$. A partir de $\mathcal{S}$, é possível construir uma matriz $M_{|V|, |\mathcal{S}|}$, na qual $m_{i,j}$ representa o estado do usuário v_i no *snapshot* s_j : 1 se $v_i \in s_j$ (“*snapshot* positivo”), ou 0 caso contrário (“*snapshot* nulo”).

Para construir traços de sessões *online* de usuários, são empregados um conjunto $V' \subseteq V$ de usuários especiais – chamados de *monitores* – para capturar *snapshots*. Um monitor $v_i \in V'$ pode contactar *bootstraps* e/ou outros usuários (ou ambos) para construir (parte de) um *snapshot*. Um *snapshot* s_t é formado pela união das listas obtidas pelos monitores: $s_t = \bigcup L(v_i, x)$. Note que o custo para obter $L(v_i, x)$ é não desprezível (por exemplo, devido ao consumo de banda, latência, frequência máxima permitida de solicitações, entre outros).

3.3. Premissas do Modelo

Duração das Sessões Online. Com base no estudo de Yao et al. [Yao et al. 2009] e Steiner et al. [Steiner et al. 2009], consideramos que quanto maior a duração de uma sessão *online* é (no passado), maior ela tende a ser (no futuro). Esse é o caso dos usuários assistindo a *streaming* de vídeo ou baixando algum conteúdo grande (e.g. a imagem de uma distribuição do GNU/Linux).

Composição de Listas de Pares. Assume-se que a probabilidade de um usuário compor uma dada lista de pares é uniforme. Essa premissa está relacionada a uma variação do problema do coletor de cupons com tamanhos de amostras aleatórias [Kobza et al. 2007], e tem sido considerada em vários casos [Hoäyfeld et al. 2011, Zhang et al. 2011]. A investigação de casos onde as listas são formadas de forma enviesadas (ou seguindo uma distribuição não uniforme) é vislumbrada como direção para trabalhos futuros.

3.4. Limitações Específicas do Sistema Alvo

Ciência de Usuários Online. Idealmente, um *snapshot* capturado no instante t deveria ser igual a $s_t \subseteq V_t$. Contudo, pode haver casos em que *snapshots* reportem incorretamente um usuário como *online*. Os motivos para isso ocorrer são vários; por exemplo, v_i pode desconectar do sistema de forma “não graciosa” (e.g. travar), ou b_k pode não receber a mensagem de desconexão de v_i . Esses casos são incomuns, podendo ocorrer devido a um aspecto específico no projeto do sistema. Por esse motivo, esse aspecto é vislumbrado como direção futura de pesquisa.

Abrangência da Lista de Pares. Idealmente, $L = V_t$ (ou $s_t = V_t$) para qualquer instante t . Contudo, limitações específicas do sistema alvo podem impedir que isso aconteça. Por exemplo, dispositivos limitados (*hardware* embarcado ou sensores) podem não ter memória suficiente para manter grandes listas de pares. Além disso, alguns sistemas limitam o tamanho da lista de pares L_{max} enviada por razões de otimização ou restrições de rede [Mansilha et al. 2010]. Pode até mesmo ser inviável para uma entidade construir e transmitir a lista completa de pares conhecidos quanto $|V_t|$ atinge a ordem de milhares ou milhões.

4. Reconstruindo Sessões de Usuários

Nesta seção apresentamos a metodologia para identificar *snapshots* imprecisos. Em seguida, descrevemos o algoritmo proposto para reconstruir traços mais acurados de sessões *online*.

4.1. Estimando a Probabilidade de Falha dos Snapshots

Assumindo que (i) a estrutura do sistema (grafo de relações entre os usuários, entre usuários e entidades de *bootstrap*, etc.) não é conhecida, e (ii) o processo de captura de *snapshots* não permite concluir qualquer informação relevante para a correção dos mesmos, nossa proposta baseia-se exclusivamente em utilizar a informação contida no conjunto de *snapshots* para estimar uma *probabilidade de falha* média p . O uso de fontes de informação específicas do sistema alvo para estimar p é vislumbrado como perspectiva de trabalho futuro.

Seja X_i o evento no qual um determinado usuário v_i está *online* e presente no *snapshot* s_j ($m_{i,j} = 1$); Y_i o evento no qual v_i não está presente em s_j ($m_{i,j} = 0$), embora *online*; e Z_i o evento no qual v_i está *offline* ($m_{i,j} = 0$). A probabilidade de termos um *snapshot* falho para v_i é dada por $P(Y_i) = p$, enquanto que a probabilidade de termos um *snapshot* correto é $P(X_i) + P(Z_i) = 1 - p$. O desafio enfrentado nesse artigo é estimar $P(Y_i)$, posto que os eventos Y_i e Z_i são indistinguíveis no *snapshot*.

Considerando p a probabilidade de um usuário falhar em aparecer em um *snapshot*, o objetivo é estimar p ($\hat{p}$) e usá-lo para corrigir o traço. Para esse fim exploramos duas observações chave. A primeira observação é que, quanto mais tempo um usuário permanece no sistema, mais se espera sua permanência [Yao et al. 2009, Steiner et al. 2009]; consequentemente, existe uma probabilidade maior de que um usuário *offline* em um *snapshot* após um período longo no sistema seja um evento do tipo Y (ao invés de Z):

Proposição 1. *Seja k uma sequência de snapshots “positivos” ($k \in \mathbb{N}^*$). A probabilidade de um snapshot “nulo” $k + 1$ ser incorreto aumenta geometricamente com k (processo de Bernoulli).*

Demonstração. Considere cada *snapshot* um ensaio de Bernoulli. Seja *falha* um ensaio no qual um determinado usuário *está online* e presente no *snapshot* (amostra positiva), e *sucesso* um ensaio no qual o mesmo usuário *está online*, mas *ausente* no *snapshot* (amostra falso negativa). Nesse caso, a probabilidade de ocorrer uma amostra falso positiva após k amostras positivas consecutivas segue um processo de Bernoulli, com uma distribuição geométrica, e é definida por $1 - (1 - p)^{k+1}$; por definição, $1 - (1 - p)^{k+1}$ aumenta geometricamente com k . $\square$

A segunda observação é que, quanto maior uma sequência de *snapshots* nulos, maior é a probabilidade de que o usuário realmente tenha saído do sistema durante aquele período:

Proposição 2. *Seja k uma sequência de snapshots nulos ($k \in \mathbb{N}^*$). A probabilidade de que todos eles sejam falhos diminui exponencialmente com k .*

Demonstração. Seja $Y_{i,j}$ o evento no qual v_i está *online* mas ausente no *snapshot* s_j , e $P(Y_{i,j}) = p$ a probabilidade de tal evento. Considerando que os usuários possuem chance uniforme de serem selecionados para compor uma lista L , temos que os eventos de falha são estatisticamente independentes. Logo, usando as regras de probabilidade condicional e da cadeia, temos

$$P\left(\bigcap_{j=1}^k Y_{i,j}\right) = \prod_{j=1}^k P\left(Y_{i,j} \mid \bigcap_{l=1}^{j-1} Y_{i,l}\right) \therefore p^n \quad (1)$$

, que por definição diminui exponencialmente com n . $\square$

Essas proposições são empregadas para se obter $\hat{p}$ com erro $1 - \alpha$, onde $\alpha \in (0, 1)$. Nesse contexto, $\alpha \rightarrow 0$ representa uma abordagem mais conservadora na busca por *snapshots* falhos, diminuindo assim a chance de falsos positivos. De maneira oposta, $\alpha \rightarrow 1$ representa uma abordagem mais agressiva, na qual mais *snapshots* são corrigidos.

Começando pela Proposição 1, que valores de p permitiriam um *snapshot* $k + 1$ ser falho para um dado usuário? Para isso, fazemos $1 - (1 - p)^{k+1} \geq 1 - (1 - \alpha)$. A probabilidade p (para k *snapshots* consecutivos) que satisfaz essa desigualdade é dada pela Equação 2.

$$p \geq 1 - (1 - \alpha)^{\frac{1}{k+1}} \quad (2)$$

Os efeitos da Proposição 2 são ilustrados na Figura 3. A probabilidade de um dado *snapshot* nulo ser um falso negativo, para um usuário v_i , decresce com a quantidade de *snapshots* nulos precedentes. Note que essa probabilidade pode ser mensurada em ambas as direções (curvas 1 e 2). Assim, para verificar se v_i tornou-se *offline* é necessário medir a probabilidade do *snapshot* s_i (mediana) ser um falso negativo. O motivo é que s_i possuirá a menor probabilidade de falha considerando a soma, nesse ponto, de ambas as curvas. Admitindo uma margem de erro de $1 - \alpha$, uma sequência k deve ser corrigida se $(1 - p)^{\frac{k+1}{2}} \geq 1 - \alpha$. A probabilidade p que satisfaz essa desigualdade é dada pela Equação 3.

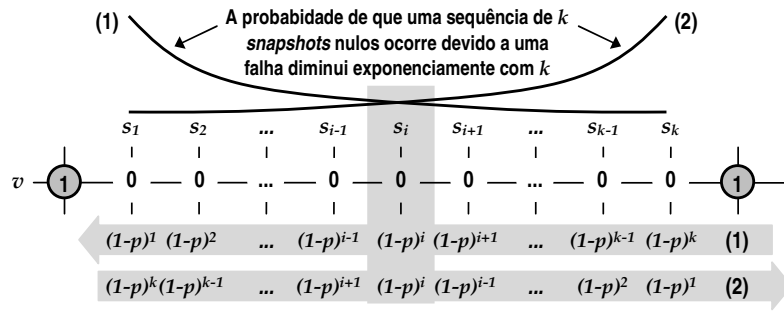


Figura 3. Probabilidade condicional

$$p \leq 1 - (1 - \alpha)^{\frac{2}{k+1}} \quad (3)$$

A probabilidade de falha para quaisquer sequências de *snapshots* é então calculada através da soma ponderada das probabilidades para cada sequência de tamanho k . Nesse caso, interessa buscar a menor probabilidade possível para essas sequências. A partir das Equações 2 e 3, define-se sistema não-linear abaixo

minimizar $\hat{p}$

$$\begin{aligned} \text{sujeito a } \hat{p} &\geq \sum_{x=1}^n P[X = x] \cdot \left(1 - (1 - \alpha)^{\frac{1}{x+1}}\right) \\ \hat{p} &\leq \sum_{y=1}^m P[Y = y] \cdot \left(1 - (1 - \alpha)^{\frac{2}{y+1}}\right) \end{aligned}$$

Nesse sistema, $P[K = k]$ é a fração (normalizada) de sequências de tamanho k , n é o tamanho da maior sequência de *snapshots* positivos, e m é o tamanho da maior sequência de *snapshots* nulos; esses valores são obtidos a partir dos *snapshots* em análise. O valor de $\hat{p}$ que satisfaz o sistema é então a probabilidade de falha nesses *snapshots*. Note que o sistema

não pode ser solucionado quando não existir um valor para $\hat{p}$ que satisfaça as duas restrições simultaneamente. Nesse caso, não há dados suficientes para estimar a probabilidade de falha.

4.2. Corrigindo Traços

Para aprimorar a acurácia de um conjunto de *snapshots* $\mathcal{S}$, basicamente aplica-se a Proposição 2 usando o valor obtido de $\hat{p}$ para as sequências de *snapshots* nulos de um dado usuário v_i . Para entender esse processo, considere um conjunto $\mathcal{S}$ dado como entrada. Baseado na metodologia descrita na subseção anterior, propõe-se os passos descritos no Algoritmo 1 para aprimorar $\mathcal{S}$.

Algorithm 1 Algoritmo para corrigir conjunto de *snapshots*

```

1: procedure CORRECTSNAPSHOTSET( $\mathcal{S}, \alpha$ ) ▷ conjunto de entrada e fator de erro
2:   pos_st.hist[] ← frequência tabulada de sequências de snapshots positivos em  $\mathcal{S}$ 
3:   nul_st.hist[] ← frequência tabulada de sequências de snapshots nulos em  $\mathcal{S}$ 
4:   pos_st.total ← número total de sequências de snapshots positivos em  $\mathcal{S}$ 
5:   nul_st.total ← número total de sequências de snapshots nulos em  $\mathcal{S}$ 
6:   pos_st.max_len ← comprimento da maior sequência de snapshots positivos em  $\mathcal{S}$ 
7:   nul_st.max_len ← comprimento da maior sequência de snapshots nulos em  $\mathcal{S}$ 
8:    $p\hat{p}os \leftarrow 0$ 
9:   for  $i \leftarrow 1 \dots pos\_st.max\_len$  do
10:     $p\hat{p}os \leftarrow p\hat{p}os + (pos\_st.hist[i] \cdot (1 - (1 - \alpha)^{\frac{1}{i+1}}))$ 
11:   end for
12:    $p\hat{n}ul \leftarrow 0$ 
13:   for  $j \leftarrow 1 \dots nul\_st.max\_len$  do
14:     $p\hat{n}ul \leftarrow p\hat{n}ul + (nul\_st.hist[j] \cdot (1 - (1 - \alpha)^{\frac{2}{j+1}}))$ 
15:   end for
16:   if  $p\hat{p}os > p\hat{n}ul$  then
17:     return null ▷ é impossível estimar a probabilidade de falha
18:   else
19:      $\hat{p} \leftarrow p\hat{p}os$ 
20:      $len \leftarrow \left\lfloor \frac{\ln(1 - \alpha)}{\ln(\hat{p})} \right\rfloor$ 
21:     for all sequências  $s_{v,1\dots n} \in \mathcal{S}$  de snapshots nulos para um dado usuário  $v_i$  do
22:       if  $length(s_{v,1\dots n}) \leq len$  then ▷ é provável que a sequência seja uma falha
23:         transforme  $s_{v,1\dots n}$  em uma sequência de snapshots positivos e substitua em  $\mathcal{S}$ 
24:       end if
25:     end for
26:     return  $\mathcal{S}$  ▷ conjunto de snapshots resultante
27:   end if
28: end procedure

```

O primeiro passo nesse procedimento (*procedure*) é obter as frequências tabuladas de tamanhos de sequências de *snapshots* positivos e nulos do conjunto de entrada $\mathcal{S}$ (linhas 2 e 3 do Algoritmo 1). Por exemplo, para o conjunto de *snapshots* apresentado na Figura 1, a frequência de sequências de *snapshots* positivos de tamanho 2 é 4 ($pos_st.hist[2] = 4$), e a de *snapshots* nulos com tamanho 5 é 1 ($pos_st.hist[5] = 1$).

O passo seguinte (linhas 4 e 5) é obter o número total de sequências de *snapshots* positivos e nulos do conjunto $\mathcal{S}$. No exemplo da Figura 1, esses totais são $pos_st.total = 7$ e $nul_st.total = 4$, respectivamente. O maior tamanho para cada uma dessas sequências também é extraído de $\mathcal{S}$ (linhas 6 e 7). No exemplo, eles são $pos_st.max_len = 7$ e $nul_st.max_len = 5$.

O próximo passo do Algoritmo 1 é estimar a probabilidade de falha para uma sequência de *snapshots* positivos ($p\hat{p}os$, linhas 8-11) e nulos ($p\hat{n}ul$, linhas 12-15) de $\mathcal{S}$, e então resolver o sistema não-linear apresentado anteriormente (linhas 16-19). Note que caso o sistema não possa ser solucionado (devido às restrições relativas a $\hat{p}$ explicadas anteriormente), então o algoritmo retorna $\mathcal{S}$, sem modificá-lo (linha 17).

Caso o sistema possua solução, o passo seguinte é determinar qual é o tamanho len da maior sequência de valores nulos que pode ser considerada uma falha (linha 20). O raciocínio por trás da equação $len = \left\lfloor \frac{\ln(1 - \alpha)}{\ln(\hat{p})} \right\rfloor$ explora a observação descrita na Proposição 2. Considerando p obtido, para que tamanhos de sequências de *snapshots* nulos as mesmas podem ser

interpretadas como falhas, admitindo um erro de $1 - \alpha$? Para responder essa pergunta, fazemos $p^{len} \geq (1 - \alpha)$. Por fim, todas as sequências de valores nulos que possuem tamanho menor ou igual a len são alteradas para sequências de *snapshots* positivos (linhas 20 - 25).

Para ilustrar esse processo, considere a Figura 1 novamente. Assumindo que $\hat{p} = 0.26$ foi obtido utilizando $\alpha = 0.95$, tem-se que $len = 2$ (linha 20). Desse modo, apenas sequências de *snapshots* nulos com tamanho menor ou igual a 2 serão corrigidas. Esse é o caso da sequência $s_{8...9}$ para o usuário v_2 . Esse usuário passará então a ter apenas uma sessão *online* no traço resultante ($s_1 - s_{14}$). Esse resultado corresponde a realidade, conforme discutido na Seção 2.

Após os *snapshots* falhos para um dado usuário v terem sido identificados e corrigidos (admitindo erro $1 - \alpha$), o conjunto de *snapshots* $\mathcal{S}$ modificado é retornado (linha 26).

5. Avaliação

Para avaliar a metodologia proposta, procurou-se responder a quatro perguntas de pesquisa: qual a efetividade da nossa metodologia na correção dos conjuntos de *snapshots*? Segundo, qual a incidência de correções falso positivas? Terceiro, quais são os cenários onde o uso da metodologia proposta é mais vantajoso? E quarto, qual o comportamento da nossa metodologia em face de diferentes configurações para α ? Para responder essas perguntas, foram executados experimentos considerando diversos conjuntos de *snapshots* e configurações. Nesta seção são discutidos os resultados mais proeminentes.

5.1. Conjunto de Dados de Referência

Nesse experimento, foi utilizado um conjunto de *snapshots* capturados de vários enxames BitTorrent[†]. Em resumo, foram empregados 50 monitores ($|V'| = 50$) para capturar um total de 2.880 *snapshots* da população de usuários *online*, durante 30 dias (iniciado no dia 15 de junho de 2013); o intervalo entre *snapshots* foi $\Delta t = 15$ minutos. No maior enxame, o número médio de usuários *online* foi 3.035, o máximo foi 5.887, e o mínimo foi 1.636. O desvio padrão foi 794,78. Cada enxame era mantido por 4 *bootstraps* (*trackers*). Uma visão parcial do conjunto de *snapshots* para esse enxame é apresentada na Figura 2(c).

O valor $|V'| = 50$ foi obtido seguindo a metodologia proposta por Hoäyfeld et al. [Hoäyfeld et al. 2011], a qual é baseada em uma variação do problema do coletor de cupons com amostras de tamanho $k > 1$ [Kobza et al. 2007]. De acordo com essa metodologia, a quantidade aproximada de respostas do *tracker* necessárias para se obter um *snapshot* completo de um determinado enxame ($E[X]$) pode ser obtida pela Equação 4.

$$E[X] \approx \frac{N \cdot h_N}{L_{max} \cdot |B|} \quad (4)$$

Na equação acima, N corresponde ao número esperado de usuários e h_N o N -simo número harmônico. A resposta do *tracker* (“lista de pares” na terminologia usada neste artigo) possui um tamanho determinado por L_{max} . Considerando uma estimativa inicial de 4.000 usuários para a população do enxame, $L_{max} = 200$ e $|B| = 4$ (ou seja, 800 usuários obtidos a cada intervalo), obteve-se $E[X] \approx 44$ monitores; para tolerar eventuais falhas, foram adicionados 6 monitores extras, totalizando assim 50 monitores.

5.2. Métricas de Avaliação

Note que um *snapshot* s_t é composto pela união de *snapshots* parciais capturados por cada monitor $v_i \in V'$. Assim, no restante desta seção denota-se como $\mathcal{S}_{|V'|}$ o conjunto de *snapshots*

[†]Informações sobre a metodologia de captura podem ser encontradas em [Mansilha et al. 2010]. O conjunto de dados usado no experimento está disponível em <http://www.inf.ufrgs.br/~wlccordeiro/idmngmt/>

capturados usando $|V'|$ monitores. Além disso, denota-se como $\mathcal{S}'_{|V'|,\alpha}$ o conjunto de *snapshots* corrigido usando a metodologia proposta, aplicando-se $\alpha = x$, e $\mathcal{S}_{gt}$ o conjunto de *snapshots* tomado como referência (*ground-truth*) para medir o desempenho da metodologia proposta. Para fins de avaliação, as seguintes métricas foram definidas:

- **Quantidade de *snapshots* positivos faltantes (ω).** Conforme a Seção 3, $M_{|V|,|S|}$ é a matriz binária construída a partir de $\mathcal{S}$. Seja $P(\mathcal{S}_{|V'|}) = \{m_{i,j} | m_{i,j} = 1\}$ o conjunto de *snapshots* positivos de $\mathcal{S}_{|V'|}$. Então, podemos definir ω por

$$\omega = |P(\mathcal{S}_{gt}) \setminus P(\mathcal{S}_{|V'|})| \quad (5)$$

Em outras palavras, ω é o número de *snapshots* positivos de $\mathcal{S}_{gt}$ (*ground-truth*) cujos correspondentes em $\mathcal{S}_{|V'|}$ (i.e. para um mesmo usuário) são *snapshots* nulos.

- **Quantidade de *snapshots* modificados (ϕ).** Essa métrica contabiliza os *snapshots* positivos em $\mathcal{S}'_{|V'|,\alpha}$ que eram inicialmente nulos no conjunto $\mathcal{S}_{|V'|}$

$$\phi = |P(\mathcal{S}'_{|V'|,\alpha}) \setminus P(\mathcal{S}_{|V'|})| \quad (6)$$

- **Quantidade de *snapshots* corrigidos corretamente (τ).** Essa métrica contabiliza os *snapshots* positivos em $\mathcal{S}'_{|V'|,\alpha}$ que, inicialmente nulos em $\mathcal{S}_{|V'|}$, coincidem com *snapshots* positivos em $\mathcal{S}_{gt}$ (*ground-truth*). Formalmente, temos

$$\tau = |(P(\mathcal{S}_{gt}) \setminus P(\mathcal{S}_{|V'|})) \cap P(\mathcal{S}'_{|V'|,\alpha})| \quad (7)$$

Os valores possíveis para τ variam no intervalo $[0, \omega]$. O valor $\tau = \omega$ é um ótimo global; significa que todos os *snapshots* foram corrigidos corretamente.

- **Número de correções falso positivas (ψ).** Uma correção falso positiva refere-se a um *snapshot* s_t que foi erroneamente corrigido para um usuário v_i . Em outras palavras, v_i de fato estava *offline* quando s_t foi capturado, mas a metodologia considerou aquele *snapshot* nulo como um caso de falso negativo. Formalmente, temos

$$\psi = |P(\mathcal{S}'_{|V'|,\alpha}) \setminus P(\mathcal{S}_{gt})| \quad (8)$$

Os valores possíveis para ψ variam no intervalo $[0, \phi]$. O valor $\psi = 0$ é um ótimo global; significa que nenhuma correção realizada foi incorreta.

5.3. Efetividade da Metodologia e Incidência de Falsos Positivos

Para avaliar a efetividade da proposta, tentou-se recriar o conjunto de *snapshots* $\mathcal{S}_{50}$ (conjunto de referência, ou *ground-truth*), tomando como entrada conjuntos de *snapshots* $\mathcal{S}_{|V'|}$ para $|V'|$ variados. Para fins de ilustração, é considerada uma visão parcial de $\mathcal{S}_{50}$ tal como mostrada na Figura 2(c). Nessa avaliação foi adotada $\alpha = 0.95$.

A Tabela 1 mostra um resumo dos resultados obtidos. Nessa tabela, a coluna *Conjunto* indica o conjunto de *snapshots* considerado. Os campos $p_{\hat{pos}}$ e $p_{\hat{nul}}$ indicam as probabilidades estimadas para as sequências de *snapshots* positivas e nulas, respectivamente. O campo *Corrigido?* indica se foi possível efetuar a correção. O campo *Comprimento* mostra o tamanho máximo para as sequências de *snapshots* nulos que foram corrigidas. Por fim, os campos *Métrica τ/ω* e *Métrica ψ/ϕ* apresentam os respectivos resultados obtidos para cada conjunto.

Avaliação dos *Snapshots* Corrigidos Corretamente. Observe na Tabela 1 que nossa metodologia alcançou alto grau de precisão na correção daqueles *snapshots* construídos com menos monitores. A proporção de *snapshots* corrigidos corretamente variou entre 65% (para $\mathcal{S}_2$) e 87% (para $\mathcal{S}_{10}$). A maior acurácia medida nos experimentos foi 90%, para o conjunto de *snapshots* $\mathcal{S}_{15}$. Não foi possível corrigir $\mathcal{S}_1$, já que $p_{\hat{pos}} > p_{\hat{nul}}$ (conforme discutido na Seção 4).

A principal conclusão que pode ser extraída da discussão acima é que a nossa metodologia alcançou melhor acurácia justamente para os conjuntos de *snapshots* construídos com

Tabela 1. Conjuntos de *snapshots* submetidos à correção, usando $\alpha = 0.95$

Conjunto	$p\hat{pos}$	$p\hat{nul}$	Corrigido?	Comprimento	Métrica τ/ω		Métrica ψ/ϕ	
					Valores	Proporção	Valores	Proporção
S_1	0.750257	0.656683	Não	–	–	–	–	–
S_2	0.720646	0.754073	Sim	9	1461/2242	.6516	31/1492	.0207
S_3	0.692409	0.817299	Sim	8	1442/1850	.7794	44/1486	.0296
S_4	0.664327	0.835471	Sim	7	1258/1617	.7779	53/1311	.0404
S_5	0.643048	0.849721	Sim	6	1107/1433	.7725	53/1160	.0456
S_6	0.623862	0.863142	Sim	6	1011/1238	.8166	61/1072	.0569
S_7	0.590806	0.868512	Sim	5	871/1023	.8514	64/935	.0684
S_8	0.555304	0.875651	Sim	5	717/814	.8808	75/792	.0946
S_9	0.543981	0.877639	Sim	4	636/741	.8582	76/712	.1067
S_{10}	0.522767	0.870914	Sim	4	540/618	.8737	80/620	.1290
S_{15}	0.473238	0.858767	Sim	4	324/360	.9000	82/406	.2019
S_{20}	0.449136	0.842509	Sim	3	193/226	.8539	81/274	.2956
S_{25}	0.420627	0.835212	Sim	3	140/159	.8805	83/223	.3721
S_{30}	0.391245	0.814221	Sim	3	70/85	.8235	83/153	.5424
S_{35}	0.337525	0.781824	Sim	2	29/35	.8285	79/108	.7314
S_{40}	0.320752	0.764094	Sim	2	14/17	.8235	79/93	.8494
S_{45}	0.302146	0.752152	Sim	2	3/5	.6000	79/82	.9634

menos recursos. Em outras palavras, nossa metodologia permite capturar traços mais acurados utilizando bem menos recursos que o estimado segundo Hoäyfeld et al. [Hoäyfeld et al. 2011].

Falso-positivos. Observe na Tabela 1 que a incidência de falso positivos é significativamente baixa. No caso de S_{10} , a ocorrência de falso positivos foi de 12%, enquanto que para S_2 a proporção foi de apenas 2%. De maneira similar à avaliação de *snapshots* corrigidos corretamente, a proporção de falso positivos diminui significativamente conforme diminui a resolução do conjunto de *snapshots*. Em resumo, nossa metodologia mostrou-se mais efetiva justamente nos cenários onde os *snapshots* foram capturados com menos recursos.

5.3.1. Avaliação dos Cenários Mais Vantajosos para Correção de *Snapshots*

A seguir é discutida uma abordagem para avaliar quais são os cenários com maior potencial para se beneficiar da nossa metodologia. Conforme discutido na Seção 4, o uso de menos monitores aumenta a probabilidade p de algum usuário *online* não ser capturado. Um limite inferior para p pode ser estimado para um sistema com população $|V|$, fotografada usando $|V'|$ monitores, e com lista de pares L_{max} , usando a Equação 9. Essa equação corresponde a uma formulação para estimar a probabilidade de falhas em um sistema ideal, com número de usuários online conhecido. Nesse caso, p corresponde à fração de usuários que não é possível se enxergar em um *snapshot*, dado um número de monitores. Considerando o conjunto de *snapshots* descrito na Seção 5.1, o limite inferior de p estimado para S_2 é 0.90, enquanto que para S_{45} é 0.0.

$$p = 1 - \frac{\min(L_{max} \cdot |V'|, |V|)}{|V|} \quad (9)$$

A Equação 9 pode ser utilizada para mensurar quando a metodologia proposta pode (significativamente) aprimorar um dado conjunto de *snapshots*. Em resumo, é possível comparar a probabilidade de falha medida $p\hat{pos}$ com o limite inferior p . Caso $p \gg p\hat{pos}$, então $S_{|V'|}$ é mais adequado para ser corrigido usando a metodologia. Isso porque a probabilidade que $S_{|V'|}$ possa ter perdido algum usuário *online* é maior do que a probabilidade estimada pela nossa metodologia. Como resultado, é possível alcançar maior precisão com a nossa metodologia do que poderia ocorrer se $p \ll p\hat{pos}$. Por exemplo, considere-se S_2 . A partir da Tabela 1, tem-se $p\hat{pos} \approx 0.72$, menor que o limite inferior $p = 0.90$, e portanto um candidato para correção com alta precisão. De maneira oposta, considere-se S_{45} . Nesse caso, tem-se $p\hat{pos} \approx 0.30$, signifi-

Tabela 2. Influência do parâmetro α na correção de $\mathcal{S}_5$

Cenário	p_{pos}	p_{nul}	Comprimento	Métrica τ/ω	Métrica ψ/ϕ
$\alpha = 0.05$	0.018886	0.038773	0	0/1433	–
$\alpha = 0.10$	0.038356	0.077806	0	0/1433	–
$\alpha = 0.20$	0.079275	0.156736	0	0/1433	–
$\alpha = 0.30$	0.123293	0.237008	0	0/1433	–
$\alpha = 0.40$	0.171157	0.318917	0	0/1433	–
$\alpha = 0.50$	0.223962	0.402891	0	0/1433	–
$\alpha = 0.60$	0.283424	0.489587	0	0/1433	–
$\alpha = 0.70$	0.352509	0.580117	1	383/1433	8/391
$\alpha = 0.80$	0.437208	0.676659	1	383/1433	8/391
$\alpha = 0.85$	0.489514	0.728632	2	656/1433	17/673
$\alpha = 0.90$	0.553874	0.784924	3	856/1433	27/883
$\alpha = 0.95$	0.643048	0.849721	6	1107/1433	53/1160
$\alpha = 0.99$	0.779671	0.922288	18	1333/1433	144/1477

cativamente maior que o limite inferior $p = 0.0$. Portanto, $\mathcal{S}_{45}$ tende a ser significativamente menos beneficiado com a metodologia (como de fato ocorreu, conforme a Tabela 1).

5.4. Análise de Sensibilidade da Metodologia

A Tabela 2 mostra os resultados de uma análise de sensibilidade tomando $\mathcal{S}_5$ como base. Observe que α regula quão conservadora nossa metodologia é ao estimar p_{pos} , e portanto definir o maior comprimento de sequências de *snapshots* nulos que serão corrigidos.

Um valor baixo para α leva a uma estimativa mais conservadora de p_{pos} . Logo, menos sequências de *snapshots* nulos serão candidatos a correção. Note que para $\alpha = 0.60$ (e valores menores), o tamanho de sequência de *snapshots* nulos que são corrigidos é 0. Isso ocorre porque a probabilidade de que apenas um único *snapshot* tenha falhado (0.28) é menor que o erro $1 - \alpha$ (ou seja, 0.40) admitido. De maneira oposta, valores mais altos para α tornam nossa metodologia mais efetiva na correção, ao custo de alguns falso positivos. Por exemplo, no caso de $\alpha = 0.70$ (e maiores), a probabilidade de falha mensurada (0.35) é maior que o erro permitido (0.30). Em resumo, ao configurar α é importante considerar quão conservador se deseja ser ao estimar a probabilidade de falha de um conjunto de *snapshots*. Em outras palavras, é necessário encontrar um balanço entre precisão (na correção) e ocorrências de falsos negativos.

6. Comportamento de Usuários em Sistemas de Compartilhamento de Arquivo

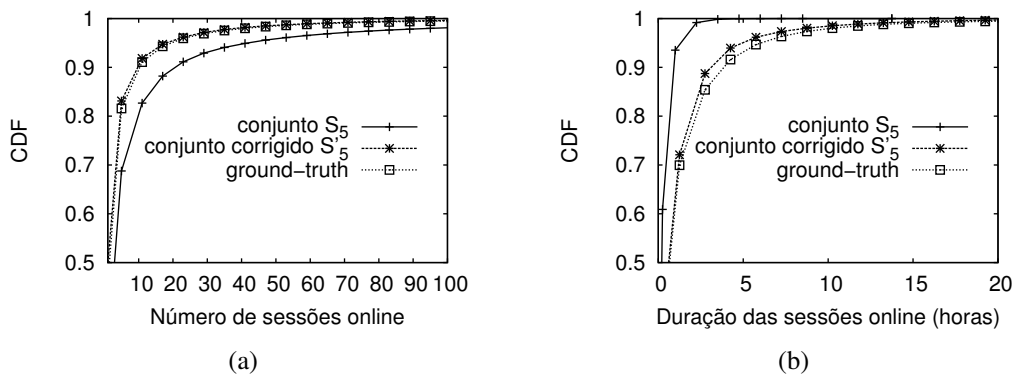
Como parte da avaliação, procurou-se responder a seguinte pergunta de pesquisa: qual o impacto da metodologia nos traços corrigidos? Para respondê-la, foi considerada uma visão completa do conjunto de *snapshots* investigado na seção anterior, com resoluções $\mathcal{S}_5$ e $\mathcal{S}_{50}$ (referência). Como métricas, foram utilizadas a recorrência de usuários (i.e., número de chegadas por usuário) e a duração das sessões *online*.

A Tabela 3 mostra um resumo estatístico para os seguintes conjuntos de *snapshots*: $\mathcal{S}_5$, $\mathcal{S}'_5$ (conjunto corrigido, usando $\alpha = 0.85$), e $\mathcal{S}_{50}$. Observe que $\mathcal{S}'_5$ é mais similar ao conjunto de referência, quando comparado com o conjunto original $\mathcal{S}_5$. Isso pode ser constatado, por exemplo, comparando o número médio de sessões por usuário e desvio padrão medido.

Um aspecto interessante a ser notado é que o número médio de sessões foi reduzido por um fator de 2 (aprox.), comparando-se os conjuntos $\mathcal{S}_5$ e $\mathcal{S}_{50}$, e que a média de duração das sessões foi aumentada por um fator de 3,5 (aprox.). Isso pode ser explicado pelo fato de que ao corrigir um intervalo de *snapshots* nulos (ou seja, alterando-se o mesmo para “positivo”) basicamente “fundem-se” duas sessões em uma. Logo, o número de sessões é decrementado por um, enquanto que a duração da nova sessão torna-se a soma da duração das sessões fundidas mais o comprimento do intervalo corrigido (que pode variar entre 1 e $\lfloor \ln(1 - \alpha) / \ln(\hat{p}) \rfloor$).

Tabela 3. Resumo das estatísticas dos conjuntos de *snapshots* estudados

Conjunto	Mín.	1º Quartil	Mediana	Média	Desvio Padrão	3º Quartil	Máx.
Número de Sessões por Usuário							
S_5	1	1	3	10.97	33.88	7	671
S'_5	1	1	2	4.95	13.45	4	303
S_{50}	1	1	2	5.36	14.83	4	552
Duração de Sessões por Usuários (minutos)							
S_5	15	15	15	28.28	25.49	30	825
S'_5	15	15	45	86.03	156.54	90	6,660
S_{50}	15	15	45	102.9	257.62	105	24,750

**Figura 4. Análise da recorrência de usuários (esquerda) e duração de sessões (direita)**

A Figura 4 mostra uma visão parcial da CDF da recorrência dos usuários e da duração das sessões. As curvas referentes a S'_5 e a S_{50} são, de maneira aproximada, iguais. Isso é uma evidência de que nossa metodologia pode aprimorar significativamente um conjunto de *snapshots* capturado com poucos recursos. Mais importante, isso representa um importante avanço sobre as soluções que formam o estado da arte, as quais requerem um número significativo de monitores para capturar *snapshots* acurados [Hoäyfeld et al. 2011, Zhang et al. 2011].

7. Trabalhos Relacionados

Ao que se sabe, nossa metodologia representa a primeira proposta formal e genérica para correção de traços. Em contraste, trabalhos anteriores tem considerado as mais variadas abordagens *ad-hoc* para correção de traços [Izal et al. 2004, Guo et al. 2005, Steiner et al. 2009]. Por exemplo, trabalhos no âmbito de BitTorrent tem considerado identificadores de sessões e informações sobre *download* e *upload* para reconstruir as sessões dos usuários [Izal et al. 2004, Guo et al. 2005]. Recentemente, Wang et al. [Wang and Kangasharju 2013] propuseram uma abordagem para calcular probabilidade de falhas em amostras de uma rede BitTorrent. No entanto, ela limita-se a estimar o número de usuários *online*, e é específica para o Mainline DHT.

Outro exemplo, mais similar à nossa metodologia, é a proposta por Steiner et al. [Steiner et al. 2009]. Nesse trabalho, o conceito de “lacunas na sessão” (noção similar ao *snapshot* nulo definido neste artigo) são identificados em um conjunto de dados, e corrigidos quando o tamanho da “lacuna” excede um limiar. Essa proposta foi usada para refatorar e analisar sessões de pares em uma rede KAD DHT. Embora essa solução possa corrigir traços de modo satisfatório, ela depende de propriedades e/ou atributos específicas do sistema alvo. Logo, ela torna-se pouco adequada para generalização e adoção em um leque maior de cenários.

Como alternativa para correção de traços, pesquisadores tem procurado superprovisionar o número de monitores dedicados para coletar *snapshots* [Hoäyfeld et al. 2011,

Zhang et al. 2011]. Embora factível, essa opção pode tornar-se extremamente cara para adoção em cenários onde a população média estimada ultrapassa a ordem de milhares ou milhões de usuários. Por exemplo, de acordo com Zhang et al. [Zhang et al. 2011], monitorar um ecossistema BitTorrent com 10.000 usuários em média requer por volta de 489 monitores; para 100.000 usuários, em torno de 6.045. Quando o sistema alvo cresce em popularidade, torna-se um desafio garantir a coordenação dos monitores, sem contar que o super-provisionamento não oferece garantias sobre a precisão dos *snapshots* capturados.

8. Considerações Finais

A correção *a posteriori* de traços de sessões *online* de usuários tem sido considerada uma estratégia promissora para melhorar a qualidade dos mesmos. Contudo, a inexistência de uma abordagem sistemática, agnóstica de propriedades específicas do sistema alvo, para corrigir traços capturados usando estratégias baseadas *snapshots* pode ter dificultado sua mais ampla adoção. Para superar essa limitação, neste artigo é proposta uma metodologia baseada em uma análise probabilística para a correção de *snapshots*.

Em resumo, nossa metodologia mostrou-se efetiva, alcançando alto grau de precisão (variando entre 65% e 90% nos cenários avaliados) na correção de *snapshots* e baixo percentual de falso positivos. Mais importante, ela gerou traços com propriedades consistentes (por exemplo, recorrência e duração das sessões *online*) quando comparado com o conjunto de referência. A importância desses resultados é ressaltada pelo fato de que capturar traços precisos requer uma fração significativa de recursos [Hoäyfeld et al. 2011, Zhang et al. 2011]. Nesse contexto, nossa metodologia representa uma importante contribuição para a comunidade, já que permite a captura de traços com grau similar de precisão enquanto utiliza muito menos recursos.

Apesar dos progressos alcançados, há muitas oportunidades de pesquisa na área. Como trabalhos futuros, pretendemos estender a metodologia para contemplar sistemas nos quais as listas de pares são construídas usando alguma heurística ou distribuição diferente da uniforme. Além disso, pretendemos investigar ajustes finos para o processo de estimativa da probabilidade de falha, tirando proveito de propriedades específicas do sistema alvo (por exemplo, tamanho de listas de pares e estimativas do número de usuários *online*), ou distribuição do tempo de permanência dos usuários [Yao et al. 2009, Steiner et al. 2009].

Agradecimentos

Os autores estão em dívida com Jair Santanna (Univ. Twente, Holanda), Jon Howell e Jay Lorch (Microsoft Research Redmond, EUA) e Alberto Montresor (Univ. Trento, Itália), pelas discussões que ajudaram a melhorar este trabalho. A pesquisa foi financiada pelo Projeto 560226/2010-1 concedido pelo CNPq (*Edital MCT/CNPq no. 09/2010 PDI – Pequeno Porte*).

Referências

- Alexa Internet, Inc. (2012). Alexa toolbar. Disponível em: <<http://www.alexa.com/toolbar>>. Acesso em: junho 2012.
- Barcellos, M. P. and Gaspary, L. P. (2006). Segurança em redes P2P: princípios, tecnologias e desafios. In *Minicursos do 24º Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos (SBRC 2006)*.
- Blond, S. L., Legout, A., Lefessant, F., Dabbous, W., and Kaafar, M. A. (2010). Spying the world from your laptop. In *3rd USENIX conference on Large-scale exploits and emergent threats (LEET'10)*, pages 1–8.
- Cuevas, R., Kryczka, M., Cuevas, A., Kaune, S., Guerrero, C., and Rejaie, R. (2010). Is content publishing in BitTorrent altruistic or profit-driven? In *6th International Conference on emerging Networking EXperiments and Technologies (Co-NEXT '10)*.

- França, T. C., Pires, P., Pirmez, L., Delicato, F. C., and Miceli, C. (2011). Web das coisas: conectando dispositivos físicos ao mundo digital. In *Minicursos do 29º Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos (SBRC 2011)*, pages 211–264.
- Guo, L., Chen, S., Xiao, Z., Tan, E., Ding, X., and Zhang, X. (2005). Measurements, analysis, and modeling of bittorrent-like systems. In *5th ACM SIGCOMM conference on Internet Measurement, IMC '05*, pages 4–4, Berkeley, CA, USA. USENIX Association.
- Guo, L., Chen, S., Xiao, Z., Tan, E., Ding, X., and Zhang, X. (2007). A performance study of bittorrent-like peer-to-peer systems. *IEEE Journal on Selected Areas in Communications*, 25(1):155–169.
- Hoäyfeld, T., Lehrieder, F., Hock, D., Oechsner, S., Despotovic, Z., Kellerer, W., and Michel, M. (2011). Characterization of BitTorrent swarms and their distribution in the Internet. *Computer Networks*, 55(5):1197–1215.
- Izal, M., Urvoy-Keller, G., Biersack, E., Felber, P., Al Hamra, A., and Garcés-Erice, L. (2004). Dissecting bittorrent: Five months in a torrent’s lifetime. In *Passive and Active Network Measurement*, volume 3015, pages 1–11. Springer Berlin Heidelberg.
- Kobza, J., Jacobson, S., and Vaughan, D. (2007). A survey of the coupon collector’s problem with random sample sizes. *Methodology and Computing in Applied Probability*, 9(4):573–584.
- Mansilha, R. B., Mezzomo, A., Facchini, G., Gaspary, L. P., and Barcellos, M. P. (2010). Observando o universo BitTorrent através de telescópios. In *28º Simpósio Brasileiro de Redes de Computadores e de Sistemas Distribuídos (SBRC 2010)*, pages 1–14.
- Rosensweig, E., Menasche, D., and Kurose, J. (2013). On the steady-state of cache networks. In *33rd IEEE International Conference on Computer Communications (INFOCOM 2013)*.
- Ruiz, L. B., Correia, L. H., and Vieira, L. F. (2006). Arquitetura de redes de sensores sem fio. In *Minicursos do 22º Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos (SBRC 2004)*, page 167–218.
- Sridharan, A. and Bolot, J. (2013). Location patterns of mobile users: A large scale study. In *33rd IEEE International Conference on Computer Communications (INFOCOM 2013)*.
- Steiner, M., En-Najjary, T., and Biersack, E. W. (2009). Long term study of peer behavior in the kad dht. *IEEE/ACM Transactions on Networking*, 17(5):1371–1384.
- Teles, A., Pinheiro, D. N., Gonçalves, J., Batista, R., Almeida, V., Endler, M., and Silva, F. J. (2013). Redes sociais móveis: conceitos, aplicações e aspectos de segurança e privacidade. In *Minicursos do 31º Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos (SBRC 2013)*.
- Wang, L. and Kangasharju, J. (2013). Measuring Large-Scale Distributed Systems: Case of BitTorrent Mainline DHT. In *IEEE Conference on Peer-to-Peer Computing (P2P 2013)*.
- Yao, Z., Wang, X., Leonard, D., and Loguinov, D. (2009). Node isolation model and age-based neighbor selection in unstructured p2p networks. *IEEE/ACM Transactions on Networking*.
- Yoshida, M. and Nakao, A. (2011a). A resource-efficient method for crawling swarm information in multiple BitTorrent networks. In *10th International Symposium on Autonomous Decentralized Systems (ISADS 2011)*, pages 497–502.
- Yoshida, M. and Nakao, A. (2011b). Measuring BitTorrent swarms beyond reach. In *IEEE International Conference on Peer-to-Peer Computing (P2P 2011)*, pages 220–229.
- Zhang, C., Dhungel, P., Wu, D., and Ross, K. W. (2011). Unraveling the bittorrent ecosystem. *IEEE Transactions on Parallel and Distributed Systems*, 22(7):1164–1177.